

Q&A Preparation Guide — Viva Voce

CMP600 Dissertation — Sergiu Ionut Pascaru

PhishGuard: A Machine Learning Approach to Real-Time Phishing URL Detection

How to use this guide: Read each question out loud, pause for 5 seconds, then answer from memory. If you cannot answer, read the model answer once and try again. Do not memorise word-for-word — understand the reasoning and speak naturally.

Part A: Research Justification

Q1. Why did you choose phishing URL detection as your topic?

Phishing is the most prevalent cyber threat globally — the APWG reported record attack volumes in 2022. Despite this, most detection systems are either reactive (blacklists) or too slow for real-time use (content-based ML). I identified a specific gap: no study had done a rigorous, controlled comparison of purely lexical classifiers on a large, modern dataset with explicit FPR reporting. This gap was both academically interesting and practically relevant.

Q2. Why did you restrict yourself to lexical features only? Isn't that limiting?

It is a deliberate constraint, not a limitation. Lexical features — extracted from the URL string alone — enable real-time detection in under 1 millisecond per URL, without any external API calls, DNS lookups, or page loading. This makes the system privacy-preserving, scalable, and deployable in environments where network access is restricted. The constraint also allowed me to isolate the contribution of structural URL patterns, which is the specific research question I was addressing. Content-based features could be added in future work.

Q3. What is the practical significance of your research?

The system provides a fast, lightweight first layer of defence against phishing. It can be integrated into browser extensions, email filters, or corporate security gateways. Because it requires no external calls, it can run entirely client-side — important for privacy-sensitive applications. The 94.03% accuracy with 5.45% FPR means that in a real deployment, only about 1 in 18 legitimate URLs would be incorrectly flagged — an acceptable rate for a first-layer filter.

Part B: Methodology

Q4. Why did you choose CRISP-DM as your methodology?

CRISP-DM is the industry-standard data mining process model, originally developed by a consortium including IBM, NCR, and DaimlerChrysler. It provides a structured, iterative framework that maps naturally to the phases of a machine learning project — from business understanding through to deployment. Mapping each CRISP-DM phase to a two-week Agile sprint allowed me to document decisions at each stage and maintain a clear audit trail, which is important for academic rigour and reproducibility.

Q5. Why did you choose an $80/20$ train/test split?

The $80/20$ split is the standard practice in supervised machine learning for datasets of this size. With 641,053 URLs, an 80% training set gives the models 512,842 examples — sufficient for learning complex patterns. The 20% test set (128,211 URLs) is large enough to provide statistically reliable evaluation metrics. I used stratified splitting to ensure both sets maintain the same $66.\overline{7}/33.3\%$ class distribution as the full dataset, preventing evaluation bias.

Q6. Why did you use stratified splitting rather than random splitting?

The dataset has a class imbalance — approximately 66.7% benign and 33.3% malicious. Without stratification, a random split could, by chance, place a disproportionate number of malicious URLs in the training set or test set, which would bias both the model and the evaluation. Stratified splitting guarantees that both sets

reflect the true class distribution, making the evaluation metrics representative of real-world performance.

Q7. Why did you use ANOVA F-test for feature selection?

ANOVA (Analysis of Variance) F-test measures the statistical relationship between each individual feature and the target class. A high F-score indicates that the feature has strong discriminative power — that is, its values differ significantly between benign and malicious URLs. I used it as a validation step after feature engineering to confirm that all 10 features I had selected were statistically meaningful. The results confirmed this — all 10 features had F-scores significant at $p < 0.001$.

Part C: Technical Decisions

Q8. Why did you choose Random Forest over other ensemble methods like Gradient Boosting or XGBoost?

Random Forest was chosen based on its established performance in the phishing detection literature — Sahingoz et al. (2019) and Rao & Pais (2020) both found ensemble methods superior for this task. Random Forest is also more interpretable than gradient boosting methods — feature importance scores are directly available, which is important for explaining predictions to end users in the PhishGuard interface. XGBoost was considered but excluded from the initial comparison to keep the study focused on the three most commonly compared classifiers in the literature.

Q9. Why did you use `class_weight='balanced'` rather than oversampling techniques like SMOTE?

Both approaches address class imbalance, but `class_weight='balanced'` is computationally simpler and does not create synthetic data points. SMOTE generates artificial minority-class examples, which can introduce noise and may not represent real phishing URL patterns accurately. The balanced class weight approach adjusts the loss function to penalise misclassification of the minority class more heavily — achieving the same goal without modifying the training data. Given the dataset already contained over 200,000 malicious URLs, synthetic oversampling was not necessary.

Q10. What does the ROC AUC score of 0.9855 mean in practical terms?

ROC AUC (Receiver Operating Characteristic Area Under the Curve) measures the model's ability to discriminate between classes across all possible classification thresholds. A score of 0.9855 means that if you randomly selected one malicious URL and one benign URL, the model would correctly rank the malicious URL as more suspicious 98.55% of the time. This is a very strong result — a perfect classifier would score 1.0, and a random classifier would score 0.5. It confirms that the Random Forest model has excellent discriminative power.

Q11. Why did you use 5-fold cross-validation rather than a simple train/test split?

Cross-validation provides a more robust estimate of model performance by training and evaluating on five different subsets of the data. This reduces the risk that the reported accuracy is an artefact of a particularly favourable train/test split. The low standard deviation in my cross-validation results — $\pm 0.17\%$ for Random Forest — confirms that the model's performance is stable and not dependent on a specific data split. I used a subsample of 100,000 URLs for cross-validation due to computational constraints, which is standard practice.

Part D: Results and Findings

Q12. Your accuracy is 94.03% — what does that mean for the 5.97% of URLs it gets wrong?

The 5.97% error rate breaks down into two types. False positives — legitimate URLs incorrectly classified as malicious — account for 5.45% of the benign class. False negatives — malicious URLs incorrectly classified as benign — account for 7% of the malicious class ($1 - \text{Recall of } 0.93$). In a real deployment, false positives are more disruptive to users, which is why I specifically tracked and reported FPR. The 5.45% FPR means approximately 1 in 18 legitimate URLs is incorrectly flagged — acceptable for a first-layer filter that would be combined with other security measures.

Q13. Why did Logistic Regression and SVM perform so similarly to each other but so differently from Random Forest?

Both Logistic Regression and LinearSVC are linear classifiers — they attempt to find a single hyperplane that separates benign from malicious URLs in the 10-dimensional feature space. They achieved similar accuracy (~78-79%) because they are both constrained by the same fundamental assumption of linearity. Random Forest, as a non-linear ensemble method, can capture complex interactions between features — for example, a URL that is both long AND has high entropy AND has many dots is far more suspicious than any single feature alone. This non-linearity is the key reason for the 15-percentage-point gap.

Q14. How does your 94.03% accuracy compare to published literature?

It is competitive. Sahingoz et al. (2019) reported 97.3% accuracy using Random Forest, but their study used a smaller dataset and included additional features beyond purely lexical ones. Ma et al. (2009) reported 95-99% accuracy but used online learning with lexical and host-based features combined. For a purely lexical approach on a large modern dataset, 94.03% is a strong result. The key differentiator of my study is the controlled comparison across three classifiers with explicit FPR reporting, which most published studies do not provide.

Part E: Limitations and Critical Thinking

Q15. What is the biggest weakness of your study?

The most significant limitation is temporal drift. The Kaggle dataset is historical, and phishing tactics evolve rapidly. A model trained on 2024 data may not perform as well on 2026 phishing URLs, particularly if attackers have adapted their URL construction patterns in response to detection systems. A production deployment would require a periodic retraining pipeline with fresh data. I acknowledge this explicitly in the dissertation because intellectual honesty about limitations is a mark of rigorous research.

Q16. Could an attacker evade your system?

Yes, a sophisticated attacker who knows the model is lexical-only could craft URLs that score low on all 10 features — for example, using a short, clean domain name with a legitimate-looking path. However, this is true of any single-layer detection system. The

PhishGuard system is designed as a first-layer filter in a multi-layered security architecture, not as a standalone solution. Combining it with reputation-based checks, content analysis, and user behaviour monitoring would significantly reduce evasion risk.

Q17. Why did you not test on a more recent dataset?

The Malicious URLs Dataset from Kaggle (Siddhartha, 2024) was the most recent large-scale, publicly available, labelled dataset I could find at the time of the study. Collecting and labelling a fresh dataset was beyond the scope and timeline of a single dissertation. This is a recognised limitation, and I recommend that future research either uses a more recent dataset or implements a temporal validation approach — training on older data and testing on newer data — to explicitly measure temporal drift.

Part F: The Application

Q18. How does the PhishGuard web application work technically?

The application is built with React on the frontend. When a user submits a URL, the JavaScript extracts the same 10 lexical features that were used to train the model — `url_length`, `domain_length`, `path_length`, `digit_count`, `letter_count`, `special_char_count`, `digit_ratio`, `dot_count`, `slash_count`, and Shannon entropy. These features are then passed to a scoring function that replicates the Random Forest model's decision logic. The risk score (0–100) is calculated from the weighted combination of feature values, and the result is displayed with a colour-coded verdict and a breakdown of each feature's contribution.

Q19. Why did you build a web application rather than just a Python script?

A web application demonstrates real-world deployment viability in a way that a Python script cannot. It shows that the model can be integrated into a user-facing product, that the feature extraction works in a browser environment without server-side computation, and that the results can be presented in an accessible, interpretable format. It also provides a concrete artefact that satisfies the deployment phase of CRISP-DM and demonstrates the practical contribution of the research.

Q20. What would you do differently if you were to repeat this study?

Three things. First, I would use a more recent dataset and implement temporal validation — training on older data and testing on newer data — to explicitly measure model decay over time. Second, I would add lightweight NLP features such as character-level *n*-grams to complement the lexical features, which would improve detection of domain-spoofing attacks. Third, I would extend the evaluation to include a user study — testing whether the PhishGuard interface effectively communicates risk to non-technical users — to validate the practical utility of the system beyond its technical performance metrics.

Quick Reference — Numbers to Know Cold

Metric	LR	RF	SVM
Accuracy	78.94%	94.03%	78.43%
Precision	0.7883	0.8946	0.7716
Recall	0.5008	0.9300	0.4984
F1-Score	0.6125	0.9120	0.6056
FPR	6.69%	5.45%	7.34%
ROC AUC	0.8241	0.9855	0.8286

Cross-validation (RF): 91.95% ± 0.17% accuracy, F1 = 0.9198 ± 0.0017

Dataset: 641,053 URLs | ⁸⁰/₂₀ stratified split | 66.7% benign / 33.3% malicious

Top 3 features: dot_count (F=31,994) · path_length (F=19,041) · slash_count (F=10,162)

Key message: Random Forest outperforms linear models by +15pp accuracy and +0.30 F1 because phishing patterns are non-linear, ensemble methods handle class imbalance better, and 200 trees capture complex feature interactions.